OSDK ARCHITECTURE & THE THREE APIS BEHIND EVERY PORTAL

Study Guide — React + OSDK, Backend SDK, Action APIs, Function APIs and the Ontology API

Week 7 · Day 1 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	1-hour concepts and architecture lecture
Audience	India and Europe batches together — this session is common to all four batches
This Week's Focus	AIP Extensibility & Enterprise Integration — exposing the Ontology and AIP capabilities beyond Foundry
This Week's Deliverable	An External Portal built with OSDK (React + Backend SDK), integrated with the batch's core enterprise systems and wired into the Executive Dashboard

Bridge From Week 6

Weeks 1–6 built, hardened and governed an insurance AI platform that people use inside Foundry: Workshop screens, AIP Logic functions, chatbots and a Production Readiness Package. Adjusters, underwriters, investigators and service reps also work in other systems, and other systems need to call your logic. This week you expose the same Ontology, Actions and functions to an external portal without giving up the governance you established.

> **Takeaway**
>
> Nothing is rebuilt. The Ontology, its Action Types and your published functions stay exactly where they are. Today's six topics are the building blocks of the portal, Day 2 adds authentication, streaming, deployment, versioning and CI/CD, and Day 3 is your self-paced build.

Learning Objectives

By the end of today, you will be able to:

- Explain how the OSDK turns chosen Ontology resources into a typed SDK, using a Developer Console application, a scoped token and the user's own permissions
- Describe how React + OSDK builds and hosts a front end: typed hooks, the shared cache, web hosting and the Workshop Iframe widget
- Describe a backend service built on the Backend SDK, with secrets kept server-side and a clear choice of whose rights each call runs with
- Apply an Ontology Action from code and validate the result, rather than trusting the HTTP status
- Execute a published function, including an AIP Logic function, from outside Foundry and treat the result as a recommendation
- Use the Ontology API to search, apply and execute over REST, and choose between it and the SDK

Topic 1 — OSDK Architecture

How Foundry's Ontology becomes a typed SDK that your own application can call.

1.1 What the OSDK Is

- **The OSDK gives your own code typed access to the Ontology —** Palantir describes it as access to "the full power of the Ontology directly from your development environment", with Foundry as the backend. For example, Batch 1's claims-status portal reads a Claim's status and writes back through one typed client, with no hand-written REST plumbing.
- **The types are generated from your Ontology, not hand-written —** Object types, properties, links, Actions and Functions you select become typed code that carries your Ontology's own property names. For example, if Claim has claimAmount and isHighValueClaim, a typo such as claimAmmount fails when the developer builds the app, not in front of an adjuster.
- **One Ontology, many languages —** TypeScript ships through npm, Python through pip or Conda, Java through Maven, and an OpenAPI specification covers any other language. For example, Batch 3's investigator portal is TypeScript and React, while its nightly Fraud Case reconciliation job is Python, and both enforce the same Action rules.

1.2 The OSDK Architecture at a Glance

The flow, in order: Ontology: objects, links, Actions, Functions → Developer Console application → Generated SDK: TS, Python, Java, OpenAPI → Your app: React front end or backend service → Users and other systems.

Reading left to right: you add Ontology resources to a Developer Console application, Foundry generates a typed SDK from exactly those resources, and your own app uses that SDK to read, call and write.

Example for Batch 2: the underwriting portal's application includes Insurance Application, Risk Assessment, the RequestRiskReview Action and the RiskScoreRefresh function. The generated SDK contains only those, and an underwriter's browser calls them through the React front end.

Every request travels back to Foundry with a scoped token, so the Ontology's own permissions still decide what each person can see and do.

1.3 What a Developer Console Application Contains

- **A custom application bundles an authorization client, SDKs, documentation, permissions and metrics —** Optional parts include web hosting, a code repository, Marketplace distribution and an MCP server. For example, Batch 2's "Underwriting Portal" application keeps its OAuth client, generated SDK and auto-generated API documentation together, so the team has one place to manage it.
- **You choose exactly which Ontology resources the SDK includes —** The SDK is generated from the resources you add, the Application overview page lists them, and an application is limited to 1,000 data resources and access restrictions combined by default. For example, Batch 4's portal adds Renewal, Retention Offer and the EscalateForCoverageRecommendation Action, and nothing from Batch 1's Claim types.
- **Generated API documentation is your first reference —** Each application's docs include install guides, code examples for loading objects and fetching pages, and references for its object types, action types and functions, with a language switch. For example, a Batch 3 developer flips the dropdown from TypeScript to Python and sees the same Fraud Case examples in the other language.

1.4 Permissions: A Scoped Token Plus the User's Own Rights

- **The token reaches only what the application was scoped to —** Access uses a token scoped only to the Ontology entities you chose, combined with the signed-in user's own permissions. For example, Batch 2's portal token cannot read Batch 1's Claim objects even for a person who can in Foundry, because Claim was never added to that application.
- **Row and column controls still apply when data is read —** Palantir states that users can never read data they are not authorized to see. For example, an underwriter who lacks the Marking on a restricted Risk Assessment gets no row back from a portal query, and the portal needs no filter code of its own to achieve that.
- **The controls do not follow the data once it leaves —** Palantir warns that these controls do not extend to the response payload or downstream processing, so pair them with Markings or Classification-based Access Control. For example, a Batch 3 portal that copies Fraud Case findings into an email or a log file has left Foundry's protection behind, and the copy needs its own handling rules.

1.5 Illustrative Portal Scope for Each Batch

Batch	Objects	Actions	Function
1 · India Claims Operations	Claim, Investigation	OpenInvestigation, ApproveClaim	FraudRiskAssessment
2 · India Underwriting & Risk	Insurance Application, Risk Assessment	RequestRiskReview, RecordUnderwritingDecision	RiskScoreRefresh
3 · Europe Fraud/SIU	Fraud Case, Evidence, Compliance Review	SubmitComplianceReview, RecordFraudOutcome	ComplianceReadinessCheck
4 · Europe Policy Servicing	Renewal, Retention Offer, Service Request	EscalateForCoverageRecommendation, RecordRenewalDecision	CoverageRecommendationAssessment

> **Takeaway**
> The OSDK is not a second data store or a second set of rules. It is a typed doorway into the Ontology you already governed in Weeks 1–6, and the doorway is only as wide as the resources you add to the Developer Console application.

Topic 2 — React + OSDK

Typed hooks, a shared cache, and a front end that talks straight to the Ontology.

2.1 What @osdk/react Provides

- **Typed hooks replace hand-written data fetching —** Hooks query objects, object sets, links and aggregations with types generated from your Ontology. For example, a Batch 1 dashboard lists

high-value claims with one hook call, instead of a custom fetch plus hand-written loading and error handling.

- **A shared, normalized cache keeps components consistent —** Objects are cached once and shared across components, and affected cached objects are synchronized automatically after an Action. For example, after an adjuster runs OpenInvestigation, the claim list and the claim detail panel both show the new status with no manual refresh.
- **Actions and Functions are first-class citizens —** You can apply and validate Actions, including optimistic updates, and call Ontology functions. For example, a Batch 4 service rep clicks escalate and the screen shows the renewal as UnderRecommendationReview immediately, then corrects itself if Foundry rejects the Action.

2.2 A Typed Hook in Practice

Illustrative, modelled on the documented useOsdkObjects pattern.

React · TypeScript

```
import { Claim } from "@my/osdk";
import { useOsdkObjects } from "@osdk/react";

function HighValueClaims() {
  const { data, error, isLoading } = useOsdkObjects(Claim, {
    where: { isHighValueClaim: true },
    orderBy: { claimAmount: "desc" },
  });

  if (data == null && isLoading) return <div>Loading...</div>;
  if (error) return <div>Error: {error.message}</div>;
  if (data == null) return <div>No claims found</div>;

  return (
    <ul>
      {data.map(c => <li key={c.$primaryKey}>{c.claimAmount}</li>)}
    </ul>
  );
}
```

Read it as three states every screen must handle: loading, error and data. The where and orderBy options are checked against Claim's generated types, so a Batch 1 developer cannot filter on a property that does not exist. The package name @my/osdk stands for your own generated SDK package.

2.3 Bootstrapping a React OSDK App

1. **Create the application and add resources**

   In Developer Console, create the application and add the objects, Actions and Functions your portal needs. For example, Batch 3 adds Fraud Case, Evidence, ComplianceReadinessCheck and SubmitComplianceReview before generating anything.

2. **Scaffold the project with the prefilled command**

   Run npm create @osdk/app@latest with your application RID, Foundry URL, client ID and SDK package name, which the Overview page prefills for you. It needs Node 18 or higher, and the CLI then asks for a project name and framework.

3. **Install, run and add the React packages**

   Run npm install and npm run dev, then add @osdk/react (and @osdk/react-components for prebuilt elements). The generated v2 client uses createClient and createPublicOauthClient, so the browser app signs in through Foundry OAuth. Day 2 covers authentication in depth.

2.4 Hosting and Embedding the React App

- **Web hosting serves a frontend-only React app from a Foundry subdomain —** Palantir's web hosting is for frontend-only apps such as React single-page apps, with no external hosting needed. For example, Batch 1's claims portal runs on a Foundry subdomain, so the team does not stand up and secure its own web server.
- **A Workshop Iframe widget can embed the app and share state both ways —** This is how the week's deliverable joins the Executive Dashboard. For example, selecting a claim on the Batch 1 dashboard can open that same claim in the embedded portal, and an action taken in the portal can refresh the dashboard.
- **Prebuilt components speed up common screens —** @osdk/react-components offers Ontology-aware interface elements that build on @osdk/react. For example, Batch 4 uses a ready-made object table for Retention Offers instead of hand-building sorting, paging and column handling.

> **Common Mistake**
>
> Using the scaffold's sign-in code and then adding secrets to it. A React app runs in the visitor's browser, so anything placed in it can be read by the visitor. Service keys and external-system credentials belong in the backend service (Topic 3).

Topic 3 — Backend SDK

Server-side code that reads, calls and writes on behalf of a user or the application.

3.1 What a Backend SDK Is

- **Python, TypeScript and Java SDKs come from the same application —** Developer Console generates them from the resources you added. For example, Batch 2's nightly job that lists stale Risk Assessments runs in Python on a server, against the same Ontology resources the React front end's types came from.
- **Two settings: running inside Foundry, or standalone —** Inside Foundry (functions, Code Workspaces, model adapters), the runtime supplies credentials and you create FoundryClient with no arguments. A standalone service supplies its own auth and the Foundry hostname. For example, Batch 3's reconciliation service runs standalone with a token held in an environment variable.
- **The backend is where secrets, retries and integration logic belong —** A browser should never hold a service secret. For example, Batch 4's backend calls an external billing service with a key kept server-side, then applies RecordRenewalDecision, and the browser only ever sees the result. Day 2 covers service-to-service authentication.

3.2 Backend SDK in Python

Illustrative, modelled on the documented FoundryClient pattern.

Python

```
import os
from ontology_sdk import FoundryClient, UserTokenAuth
from ontology_sdk.ontology.objects import Claim

auth = UserTokenAuth(token=os.environ["FOUNDRY_TOKEN"])
client = FoundryClient(auth=auth, hostname="<YOUR-FOUNDRY-URL>")

# Filter, then iterate every match
high_value = client.ontology.objects.Claim.where(
    Claim.object_type.claim_amount > 25000
).iterate()

for claim in high_value:
    # One-to-many link: returns an object set
    for inv in claim.investigations().iterate():
        print(claim.claim_id, inv.risk_level)
```

The token comes from an environment variable and never from source code. where narrows the set on the server, iterate pages through every match, and the link call returns a set you iterate in turn. The generated package name (ontology_sdk here) comes from your own application.

3.3 Backend SDK in TypeScript

Filtering, link traversal and aggregation.

TypeScript

```
// A page of high-value claims
const page = await client(Claim)
  .where({ claimAmount: { $gt: 25000 } })
  .fetchPage({ $pageSize: 30 });

// Traverse the link to Investigations
const investigations = client(Claim)
  .where({ isHighValueClaim: true })
  .pivotTo("investigations");

// Count without loading every object
const result = await client(Claim)
  .aggregate({ $select: { $count: "unordered" } });
// read it as result.$count
```

pivotTo moves from a set of Claims to the set of their linked Investigations, and aggregate returns a number without pulling objects over the network. For example, a Batch 1 manager's tile showing “claims over 25,000” should use the aggregate, not load thousands of claims to count them.

3.4 Python, TypeScript, Java or OpenAPI?

Choice	Good fit when	Batch example
TypeScript	The team already writes the React front end and wants one language across front and back	Batch 1 claims portal with a small Node backend
Python	Data engineers own the service, or it runs inside Foundry or a Code Workspace	Batch 2 nightly stale-score report

Choice	Good fit when	Batch example
Java	The core enterprise system is already a Java service	Batch 4 policy administration integration
OpenAPI	No generated SDK exists for the language, or a quick tool needs raw calls	Batch 3 case-management tool in another language

3.5 Permission-Aware Backend Services

- **Call with the user's token when the answer depends on who is asking —** Foundry then enforces what that person may see. For example, a Batch 3 portal that shows each investigator only their own Fraud Cases should pass the investigator's token, so no filtering code in your own service decides who sees what.
- **An application identity is for jobs that no single user owns —** Scheduled reconciliation and batch jobs have no signed-in person. For example, Batch 4's nightly check of expiring Retention Offers runs as the application, with the narrowest scope that does the job. Day 2 covers how that identity is configured.
- **Never widen access in your own code to make something work —** Caching a restricted result server-side and serving it to everyone bypasses the controls Foundry enforced at read time. For example, storing a Batch 2 Risk Assessment under a Marking in a shared cache lets an unauthorized underwriter read it.

> **Platform Mechanic**
>
> The generated package name, client class and method names come from your own Developer Console application. The Python and TypeScript samples in this guide follow Palantir's documented patterns with this program's object names substituted in, so use your own application's API documentation for the exact names.

Topic 4 — Action APIs

Writing back to the Ontology through the same governed Action Types.

4.1 What the Action API Is

- **Calling an Action from code runs the same Action Type —** Its rules, submission criteria and permissions apply whoever calls it. For example, Batch 1's OpenInvestigation refuses a claim that fails its submission criterion whether it is clicked in Workshop, proposed by the chatbot or sent from the portal.
- **Apply one, or apply a batch that succeeds or fails together —** The SDKs offer applyAction and batchApplyAction, and the batch is all-or-nothing. For example, Batch 4's backend submits RecordRenewalDecision for 40 renewals in one batch, and if a single renewal is invalid none of the 40 are written.
- **You decide whether edits come back —** Requesting the edits returns what the Action changed. For example, the Batch 3 portal asks for edits after SubmitComplianceReview and shows “Compliance Review created” with a link to the new object.

4.2 Applying an Action

Illustrative, modelled on the documented applyAction and Python patterns.

TypeScript

```
const result = await client(openInvestigation).applyAction(
  { claimId: "<claimId>" },
  { $returnEdits: true },
);

if (result.type === "edits") {
  console.log("Investigation opened", result);
} else {
  console.log("Validation failed", result);
}
```

Python

```
from foundry_sdk_runtime.types import (
    ActionConfig, ActionMode, ReturnEditsMode)

response = client.ontology.actions.open_investigation(
    action_config=ActionConfig(
        mode=ActionMode.VALIDATE_AND_EXECUTE,
        return_edits=ReturnEditsMode.ALL),
    claim_id="<claimId>",
)
if response.validation.result == "VALID":
    print(response.edits)
```

Both versions branch on the validation outcome rather than assuming success. The parameter names are illustrative: your own application's generated documentation shows the exact names and types for OpenInvestigation.

4.3 Validation Is the Answer, Not the HTTP Status

- **A 200 response means the request was processed, not that the Action applied —** Palantir's REST reference says to check the validation result to confirm the Action was applied. For example, ApproveClaim returns 200 but validation fails because the claim is no longer eligible, and a portal that checks only the status code would tell the adjuster it worked.
- **Show the person why an Action was refused —** A silent failure sends users into retry loops. For example, when Batch 2's RequestRiskReview fails its submission criterion, the portal shows the reason and the underwriter knows to refresh the record instead of clicking again.
- **Duplicate-write protection is your design, not an automatic feature —** The apply endpoint's reference documents no idempotency key, so a repeat is stopped by your Action's own criteria and your app. For example, a double-click sends SubmitComplianceReview twice, and the second is rejected because the first moved the case to PendingRegulatoryReferral. Day 2 returns to this.

4.4 Action API Across the Four Batches

Batch	Action	What stops a repeated call
1 · Claims	OpenInvestigation	The Action's submission criterion on the claim's current state
2 · Underwriting	RequestRiskReview	The Action's own submission criterion on the application or its assessment

Batch	Action	What stops a repeated call
3 · Fraud/SIU	SubmitComplianceReview	Criterion: status Open or UnderReview; the first call sets PendingRegulatoryReferral
4 · Policy Servicing	EscalateForCoverageRecommendation	Criterion: renewal status Pending; the first call sets UnderRecommendationReview

> **Common Mistake**
>
> Reporting success to a user because the call returned without an exception. Read the validation result (and the edits, if requested) before telling an adjuster, underwriter, investigator or service rep that an Action happened.

Topic 5 — Function APIs

Calling published functions, including your AIP Logic functions, from outside Foundry.

5.1 What the Function API Is

- **Executing a function is one typed call —** TypeScript uses executeFunction and Python uses client.ontology.queries with keyword arguments only. For example, Batch 1's portal calls FraudRiskAssessment with a claimId and receives fraudRiskTier and recommendedFraudAction as typed fields.
- **Add the function to the application first —** The SDK contains only the resources you added, and Functions are among them. For example, if FraudRiskAssessment was never added in Developer Console, it will not exist in Batch 1's generated SDK. Confirm your published AIP Logic function appears in your own resource picker.
- **Results come back typed —** Functions can return objects, object sets, aggregations and other typed values, depending on how they were published. For example, CoverageRecommendationAssessment returns recommendationTier and findings that the Batch 4 portal reads by name.

5.2 Executing a Function

Illustrative, modelled on the documented executeFunction and queries patterns.

TypeScript

```
const result = await client(fraudRiskAssessment)
  .executeFunction({ claimId: "<claimId>" });

// e.g. fraudRiskTier, findings,
// recommendedFraudAction
console.log(result);
```

Python

```
result = client.ontology.queries.fraud_risk_assessment(
    claim_id="<claimId>"
)

# keyword arguments only
print(result)
```

Both calls pass exactly one input, the claimId. Treat the result as a recommendation to show, not an instruction to execute: the write happens only through an Action call.

5.3 Calling FraudRiskAssessment From a Backend Service

1. **The portal sends the claimId to the backend, which executes the function**
   The LLM-backed block runs, so this call costs tokens and takes seconds, which is why it runs for the one claim an adjuster opens, never for every row in a list.
2. **The backend returns the tier, findings and recommended action as a proposal**
   The screen shows fraudRiskTier, findings and recommendedFraudAction = OpenInvestigation as a suggestion, the same human-in-the-loop principle as the chatbots in Week 5.
3. **The adjuster confirms, and the backend applies the Action**
   Only the confirmation triggers the OpenInvestigation Action call, so Foundry's submission criteria and audit trail govern the write, and the function itself never changes data.

5.4 Production Concerns When Calling Functions

- **Choose which function version your app calls —** The execute endpoint accepts a semantic version range and defaults to the most recently published version. For example, pinning Batch 2's portal to a tested RiskScoreRefresh range means next week's prompt change does not alter portal behavior until the team has re-run its evaluation suite.
- **Every LLM-backed call has cost and latency —** This is Week 6's cost-management and capacity-planning lesson, now with an external caller. For example, a Batch 3 list page that ran ComplianceReadinessCheck for 50 cases on load would multiply tokens fiftyfold, and it should run only when an investigator opens one case.
- **Read data with the Ontology, call a function only for judgment —** Plain object reads are cheaper, faster and repeatable. For example, the portal shows claimAmount by reading the Claim object, and calls FraudRiskAssessment only for the fraud-pattern read, the same deterministic-versus-LLM rule from Week 5.

> **Takeaway**
> A function recommends, an Action writes. Keeping that split means an LLM-backed result never changes data by itself, and every change flows through the Action Type's criteria, permissions and audit trail.

Topic 6 — Ontology API

The REST layer underneath the SDKs, for any language and any quick integration.

6.1 What the Ontology API Is

- **A REST interface to the same Ontology the SDKs use —** Palantir describes the OSDK as ergonomic access to the Ontology APIs, so the REST API is the layer beneath it. For example, a Batch 2 integration tool written in a language with no generated SDK reads Risk Assessments over plain HTTPS.
- **Three families of calls: objects, Actions and queries —** Objects can be listed, fetched, searched and aggregated, with linked objects reachable too. Actions are applied singly or in batches, and queries (your Functions) are executed. For example, Batch 3's reporting tool uses object search, and its case-closing service uses the Action apply call.
- **A bearer token with scopes —** Requests carry Authorization: Bearer with a token, and third-party OAuth2 applications need api:ontologies-read to read and api:ontologies-write to apply Actions. For example, a read-only Batch 4 report integration is issued the read scope alone.

6.2 Three Core Endpoints (v2)

Purpose	Method and path	Notes
Search objects	POST /api/v2/ontologies/{ontology}/objects/{objectType}/search	Body can carry where, orderBy, pageSize, pageToken and select
Apply an Action	POST /api/v2/ontologies/{ontology}/actions/{action}/apply	Body carries parameters; check the validation result in the response
Execute a query	POST /api/v2/ontologies/{ontology}/queries/{queryApiName}/execute	Body carries parameters; the answer arrives in value; optional version range

6.3 Raw REST Calls

Illustrative requests; confirm exact filter and parameter shapes in your API reference.

Search claims over 25,000

```
URL="https://<HOST>/api/v2/ontologies/<ontology>"

curl -X POST "$URL/objects/Claim/search" \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-type: application/json" \
 -d '{ "where": { "type": "gt",
                  "field": "claimAmount",
                  "value": 25000 },
       "pageSize": 30 }'
```

Execute FraudRiskAssessment

```
URL="https://<HOST>/api/v2/ontologies/<ontology>"

curl -X POST \
 "$URL/queries/fraudRiskAssessment/execute" \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-type: application/json" \
 -d '{ "parameters": { "claimId": "<claimId>" } }'

# response shape: { "value": ... }
```

The same request shape applies to the apply call: a parameters map, a bearer token and a JSON content type. Read the response, not just the status.

6.4 Paging, Consistency and Limits

- **Page with nextPageToken until it stops coming back —** Omit pageToken on the first call and pass the previous response's nextPageToken afterwards, because a page may hold fewer objects than pageSize. For example, a Batch 1 export of claims keeps requesting until no token returns, instead of assuming 30 rows means the end.
- **Ask for a snapshot when a consistent view matters —** Without it, you may see duplicates or missing items while paging, and relevance ordering can repeat or skip objects across pages. For example, Batch 3's month-end Fraud Case count uses a snapshot, so a case that changes mid-export is not counted twice.

- **Mind the filter limits —** Search queries can be nested at most three levels deep, and partial terms do not match term filters except where documented. For example, a Batch 2 search for “Underwr” will not match “Underwriting” through a term filter, so the portal uses the documented prefix-aware filter instead.

6.5 OSDK or Raw REST?

Situation	Prefer	Why
React front end	OSDK with @osdk/react	Typed hooks, shared cache and Action sync come for free
Python or TypeScript backend	Generated SDK	Compile-time checks on property and Action names
Language with no SDK, or a throwaway script	Ontology API directly	No generation step, but you handle types and paging yourself
Connecting a third-party tool	Ontology API with a scoped token	The tool only needs the three endpoint families it uses

> **Platform Mechanic**
>
> The three endpoint paths in this guide are taken from Palantir's v2 API reference. The filter body in the search example is illustrative: confirm the exact shape of where clauses and the available options in the reference for your Foundry version before relying on them.

Common Pitfalls

- **Putting a service secret or a user token into browser code —** A Batch 4 front end that embeds the billing service key exposes it to every visitor. Secrets and service calls belong in the backend, and the browser should only hold a signed-in user's own session.
- **Treating a 200 response as proof an Action was applied —** ApproveClaim (Batch 1) can return 200 while its validation fails because the claim is no longer eligible. Always branch on the validation result and show the person the outcome.
- **Running an LLM-backed function once per row of a list —** Executing ComplianceReadinessCheck (Batch 3) for 50 cases on page load multiplies tokens and latency fiftyfold. Run it on demand for the one case a person opens.
- **Adding every Ontology resource to the application “just in case” —** A Batch 2 portal that includes Claim types widens what its token can reach for no benefit. Scope the SDK to exactly what the portal uses, and revisit the list as the app changes.
- **Assuming Foundry's read controls follow data after it leaves —** Copying a restricted Risk Assessment (Batch 2) into a log or an email puts it outside its Marking. Treat every copy as needing its own handling rule.
- **Assuming defaults you relied on elsewhere carry over to an API call —** Palantir's apply endpoint reference says parameter default values are not supported there. Check your application's generated documentation for exactly what a caller must send, instead of assuming Workshop or chatbot defaults apply.

Glossary

Term	Meaning
OSDK	Ontology SDK: typed code generated from the Ontology resources you add to a Developer Console application, available for TypeScript, Python and Java, with an OpenAPI specification for other languages
Developer Console application	The unit that bundles an authorization client, generated SDKs, API documentation, permissions and metrics, with optional web hosting
@osdk/react	Typed React hooks and a shared, normalized cache for building front ends on the OSDK
Backend SDK	The generated Python, TypeScript or Java SDK used from a server-side process, where secrets and integration logic belong
Action API	Applying an Ontology Action Type from code, singly or as an all-or-nothing batch, with the validation result as the real outcome
Function API	Executing a published function, including an AIP Logic function, from code, with optional version selection
Ontology API	The REST interface beneath the SDKs: search objects, apply Actions and execute queries, using a bearer token and OAuth scopes
Scoped token	A token limited to the Ontology entities an application was configured for, combined with the signed-in user's own permissions
Snapshot	An option on object search that gives a consistent view while paging through results

Reflection Questions

- List the objects, Actions and functions your batch's portal needs. Which of them did you almost add “just in case”, and what would that have widened?
- Pick one screen in your portal. Which parts are a plain object read, and which, if any, genuinely need an LLM-backed function?
- For one Action in your batch, what stops a double-click from writing twice, and where does that guard live?
- Which calls in your portal should run with the signed-in user's token, and which, if any, should run as the application?
- What would a read-only third-party integration for your batch need in terms of endpoints and scopes?

Further Practice

Before Day 2, open the API documentation of your own Developer Console application (or, if it is not created yet, the documentation page Palantir publishes for the OSDK) and find the exact names for one object type, one Action and one function from your batch. Day 2 covers authentication, streaming, deployment, versioning and CI/CD, and Day 3 is your self-paced portal build.